Deliverable 2: Summary of Methods and Findings for Task 4

For this project, we designed both a binary classification model and a multi-class prediction model. Before designing the models, we ran some preprocessing on our data. First, we tokenized the websites, converting them from long strings ot texts into sets of words. We also removed stop words and words shorter than three letters (such as I, me, and is, for example). After that, we used lemmatization to merge words that shared the same base, such as run and running, into just one word. We then converted the list of words from each document into a matrix showing the frequency of each word within each document. Finally, we filtered this matrix to remove rare words that appeared in fewer than seven documents. This helps prevents overfitting and improves the efficiency of our model.

When doing our initial training and testing split, we wanted to train the models on the same data and test the models on the same data. To do this, we stratified the dataset based on the multi-class categories for webpages. We chose to split the data this way since the multi-class categories for webpages within the dataset are in fact subcategories of the ones used for the binary classification. Doing this maintains the same split of the data overall for both binary and multi-class groupings for when we trained our models later on.

For our first predictive model, the binary classification model, we used a support vector machine. This model draws a hyperplane between the data to separate it into two categories, those being relevant claim content and no relevant claim content category. To optimize our hyperplane, we used cross-validation with 5 folds to determine the cost value, which represents how much the model should penalize any improperly classified data points or data points that fall within the margin when generating the hyperplane. The final performance of this model was fairly solid, with an accuracy of 0.8055556, sensitivity of 0.7845304, specificity of 0.8232558, and a ROC AUC value of 0.8801105.

For our second predictive model, the multi-class model, we used a random forest algorithm. This uses a set of decision trees to categorize each webpage based on the frequency of certain keywords within the page. After optimizing the model, we decided that the best version of it consists of 500 trees and uses 55 predictor words from the data. The final performance of this model was also fairly solid, with an accuracy of 0.8308081, sensitivity of 0.7584945, specificity of 0.9475951, and an average one-to-all ROC AUC value of 0.9581900.